CHAPTER 7

Using Test Doubles as Mocks and Stubs

You have a problem. You want to add credit-card processing to your project application so that you can make money. Testing the credit-card functionality presents immediate difficulties. For one thing, you don't want to accidentally make a credit-card purchase during testing—that would be bad. But even if the purchase gateway provides a test sandbox, you still don't want to depend on it for your unit tests to run. That network call is slow and you don't want your passing tests to depend on the status of a remote server.

Or, you might have a different problem. You'd like to build your code using a modular design. In doing so, you'd like your tests to be as isolated as possible from dependencies on other parts of the code. For example, you might have business logic that calls a model but that you want to test without depending on the model. You want your workflow test to work even if the model is broken—even if the model code does not yet exist.

The solution to both of these problems is a *test double*. A test double is a "fake" object used in place of a "real" object for the purposes of automated testing. By "fake," I mean that the object doesn't have a real implementation, but might instead have canned values that it can return in response to specific messages.

You might use a double when the real object is unavailable or difficult to access from a test environment. Or you might use a double to create a specific application state that would otherwise be difficult to trigger in a test environment. Database or network failures, for example, might depend on program states that are nondeterministic or otherwise hard to replicate in testing.

Doubles can also be used strategically to limit a test's execution to the object and method specifically under test, or to specify the interface of an object or part of an object that does not yet exist. Doubles can be used to support a